

UAPOML — Uncertainty-Aware Portfolio Optimization using Machine Learning

Week 2 Problem Set: Portfolio Management & Machine Learning

Topics: Portfolio Metrics · KNN · Linear Regression · Backtesting · Risk Analysis

Total Marks: 100

Libraries: NumPy, Pandas, Matplotlib

Language: Python 3.x

Submission: Jupyter Notebook (.ipynb)

Instructions:

- Answer all 10 problems. Clearly label each answer with the problem number.
- Show all intermediate steps, formulas, and code. Unexplained answers may receive **0 marks**.
- For all plots: add title, labelled axes, and a legend where applicable.
- Do **not** use `scikit-learn` for Problems 7–9 (implement from scratch using NumPy/Pandas).
- You *may* use `scikit-learn` only in the final sub-part of Problem 9 for verification.

Section A — Portfolio Fundamentals & Risk Metrics (40 marks)

Problem 1

Portfolio Construction & Daily Returns

You are given the following daily closing prices (in ₹) for 4 assets over 6 trading days:

Asset	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6
RELIANCE	2800	2850	2830	2900	2880	2950
INFY	1450	1470	1460	1490	1510	1500
HDFCBANK	1600	1580	1610	1625	1615	1640
TATAMOTORS	520	535	528	540	555	548

You invest **10,00,000** with equal weights ($w_i = 0.25$ each).

- Using Pandas, create a `prices_df` DataFrame from the table above, compute daily **percentage returns**, and display the returns matrix.
- Compute the **units purchased** for each asset on Day 1 and calculate the **daily portfolio value** for all 6 days using a vectorised approach (no loops). Plot the portfolio value over time.

- c) Compute the **portfolio's annualised volatility** assuming 252 trading days.
Use: $\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$

Hint / Context

Recall from the lecture: `daily_portfolio_values = prices_df.dot(units)` computes the portfolio value in a single line using matrix multiplication.

Problem 2

Risk Metrics — VaR, CVaR, and Drawdowns

Using the **same portfolio** from Problem 1 (extend to at least 50 simulated trading days if needed — generate synthetic returns using `numpy.random.normal` with $\mu = 0.001$, $\sigma = 0.015$):

- a) Compute the **Historical Value at Risk (VaR)** at confidence levels of 95% and 99%.

$$\text{VaR}_\alpha = -\text{percentile}(R, 1 - \alpha)$$

Interpret what each value means in plain language.

- b) Compute **Conditional VaR (CVaR)** — the average loss on days *worse* than the VaR threshold — for both confidence levels. Why is CVaR considered more informative than VaR for tail risk?

- c) Calculate the **Maximum Drawdown (MaxDD)** of the portfolio:

$$\text{MaxDD} = \frac{\text{Peak} - \text{Trough}}{\text{Peak}}$$

Plot the **drawdown curve** over time, shading the maximum drawdown region.

Hint / Context

Drawdown at time t : $DD_t = \frac{\max_{s \leq t}(V_s) - V_t}{\max_{s \leq t}(V_s)}$, where V_t is the portfolio value at time t .

Problem 3

Risk-Adjusted Performance Metrics

Using 252 days of synthetic portfolio returns and a risk-free rate of $R_f = 6\%$ per annum ($\approx 0.06/252$ per day):

- a) Compute the **Sharpe Ratio**:

$$\text{Sharpe} = \frac{R_p - R_f}{\sigma_p}$$

Annualise both the return and volatility before computing.

- b) Compute the **Sortino Ratio** using only *negative* returns for the downside deviation σ_d :

$$\text{Sortino} = \frac{R_p - R_f}{\sigma_d}$$

Why does Sortino penalise the portfolio less than Sharpe when upside volatility is high?

- c) Compare both ratios in a formatted Pandas DataFrame and comment on which metric is more appropriate for an asymmetrically returning strategy.

Problem 4

Backtesting a Simple Moving Average Strategy

Using 200 days of price data for a single asset (generate with a random walk: $P_t = P_{t-1}(1 + \epsilon_t)$, where $\epsilon_t \sim \mathcal{N}(0.001, 0.018)$, $P_0 = 1000$):

- a) Implement a **SMA Crossover Strategy**: buy when the 10-day SMA crosses above the 30-day SMA; sell (or short) when it crosses below. Generate a **signal** column (+1 for long, -1 for short, 0 for hold).
- b) Simulate the strategy over the 200 days with initial capital 1,00,000. Compute:
- Strategy cumulative return
 - Buy-and-hold cumulative return (as benchmark)

Plot both on the same graph.

- c) Compute the **Win Rate** and **Profit Factor** for the strategy:

$$\text{Win Rate} = \frac{\text{Winning Trades}}{\text{Total Trades}}, \quad \text{Profit Factor} = \frac{\text{Total Profit}}{\text{Total Loss}}$$

Does a Profit Factor > 1 guarantee a good strategy? Explain briefly.

Hint / Context

Use `pd.Series.rolling(window).mean()` to compute moving averages. Detect crossovers by comparing `signal.diff()`.

Section B — Machine Learning Applications

(45 marks)

Problem 5

Feature Engineering for ML Models

This is a preparatory problem shared by Problems 6–9. Using a simulated price series of 300 days (same random walk as Problem 4, $P_0 = 500$):

- a) Engineer the following features using Pandas and NumPy (no loops):

- **Return_1d**: 1-day percentage return r_t
- **SMA_5, SMA_20**: 5-day and 20-day simple moving averages
- **Volatility_10**: Rolling 10-day standard deviation of returns
- **Momentum_5**: $P_t - P_{t-5}$ (5-day price momentum)

Drop rows with NaN and reset the index.

b) Create a binary **target variable**:

$$y_t = \begin{cases} 1 & \text{if } r_{t+1} > 0 \text{ (price rises tomorrow)} \\ 0 & \text{otherwise} \end{cases}$$

What is the class balance? Visualise it with a bar chart.

c) Normalise all features using **Min-Max scaling** (implement manually using NumPy):

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Why is feature scaling critical before applying KNN?

Problem 6

K-Nearest Neighbours (KNN) — From Scratch

Implement a **KNN Classifier from scratch** using only NumPy (no `scikit-learn`).

a) Write a function `euclidean_distance(x1, x2)` that computes:

$$d(\mathbf{x}_1, \mathbf{x}_2) = \sqrt{\sum_{j=1}^p (x_{1j} - x_{2j})^2}$$

Then write `knn_predict(X_train, y_train, X_test, k)` that:

- Computes distances from each test point to all training points
 - Selects the k nearest neighbours
 - Returns the majority vote label (1 or 0)
- b) Using the features and target from Problem 5 (80/20 train-test split), predict next-day price direction with $k \in \{3, 5, 7, 11, 15\}$. Report the **accuracy** for each k and plot an *Accuracy vs. k* curve. Which k is optimal?
- c) Compute the **Confusion Matrix** for the best k :

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

In the context of trading, which metric (Precision or Recall) is more important when false positives are costly trades? Justify your answer.

Hint / Context

Use `np.argsort(distances)[:k]` to find the indices of the k nearest neighbours.

Problem 7*Linear Regression — Return Prediction from Scratch*

Implement **Linear Regression from scratch** using the Normal Equation.

- a) Given features $\mathbf{X}$ and continuous target $\hat{r}_{t+1}$ (next-day return, not binary), derive and implement the closed-form solution:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Prepend a bias column of ones to $\mathbf{X}$ before computation. Print the learned coefficients and interpret the sign of each feature's coefficient.

- b) Predict returns on the test set. Compute:

- **Mean Squared Error (MSE)**: $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- **R² Score**: $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

Plot *Actual vs. Predicted Returns* as a scatter plot with the ideal $y = x$ line overlaid.

- c) **Gradient Descent implementation**: Alternatively, implement Linear Regression using **Batch Gradient Descent**:

$$\beta \leftarrow \beta - \frac{\eta}{n} \mathbf{X}^T (\mathbf{X}\beta - \mathbf{y})$$

Use $\eta = 0.01$ and 1000 iterations. Plot the **loss curve** (MSE vs. iteration). Do the coefficients converge to the Normal Equation solution?

Problem 8*Portfolio Allocation with Linear Regression*

Bridge ML predictions with portfolio construction.

- a) Using your Linear Regression model from Problem 7, predict the **expected returns** $\hat{\mu}_i$ for each of the 4 assets from Problem 1 (re-engineer features for each). Arrange predictions in a NumPy array.
- b) Assign portfolio weights **proportional to predicted returns** (only allocate to assets with positive predictions):

$$w_i = \frac{\max(\hat{\mu}_i, 0)}{\sum_j \max(\hat{\mu}_j, 0)}$$

Verify that weights sum to 1.

- c) Backtest this *ML-driven allocation* for the available test period and compare cumulative returns with equal-weight allocation using a line plot. Does the ML-driven portfolio outperform? Discuss one limitation of this approach.

Problem 9*Model Comparison & Hyperparameter Tuning*

- a) Implement **k-Fold Cross Validation** (with $k = 5$, using NumPy array splitting — no `scikit-learn`) to evaluate your KNN classifier's accuracy for $k_{\text{KNN}} \in \{3, 7, 11\}$. Report the mean and standard deviation of accuracy across folds.
- b) Summarise the performance of **KNN Classifier** and **Linear Regression** in a Pandas DataFrame with columns:

[Model, Accuracy/R², MSE/N.A., Best Param]

- c) Based on the table, which model would you deploy in a real trading system and why? Discuss at least two risks of deploying an ML model in a live portfolio (e.g., overfitting, look-ahead bias).

Section C — Conceptual & Critical Thinking**(15 marks)****Problem 10***Theory, Design, and Reflection*

Answer the following short questions. Each answer should be **3–6 sentences**.

- a) **Diversification:** Harry Markowitz famously said “*Diversification is the only free lunch in finance.*” Using the concept of portfolio variance:

$$\sigma_p^2 = \sum_i w_i^2 \sigma_i^2 + \sum_{i \neq j} w_i w_j \sigma_i \sigma_j \rho_{ij}$$

Explain mathematically *why* combining assets with low (or negative) correlation ρ_{ij} reduces total portfolio risk.

- b) **Technical vs. Fundamental Analysis in ML:** KNN is essentially a *technical* approach (it looks at past numerical features without understanding fundamentals). Propose **one feature derived from fundamental analysis** (e.g., P/E ratio, debt-to-equity) that could be added to your KNN feature set. How would you obtain and integrate it?
- c) **The Curse of Dimensionality:** As you add more features to KNN, explain what happens to the concept of “nearest neighbours.” How does this affect the model's performance in high-dimensional financial data? Suggest one technique to mitigate this.
- d) **Overfitting in Backtesting:** A strategy achieves a Sharpe Ratio of 3.5 in backtesting but 0.4 in live trading. List **three specific causes** of this gap (e.g., look-ahead bias, survivorship bias, data snooping) and suggest one validation method to detect each.

- e) **Linear Regression Assumptions:** Linear Regression assumes that residuals are independently and identically distributed (i.i.d). Financial return data often violates this. Name **two specific violations** common in financial time series and briefly describe how they could distort your model's predictions.

Grading Summary		
Section	Topic	Marks
A	Portfolio Fundamentals & Risk Metrics (Problems 1–4)	40
B	Machine Learning Applications (Problems 5–9)	45
C	Conceptual & Critical Thinking (Problem 10)	15
Total		100

Uncertainty-Aware Portfolio Optimization using ML — Week 2 · All the best!
 $\mathcal{O}(\text{profit}) \gg \mathcal{O}(\text{loss})$